# Bank Account Simulation

ACProgramming Mini-Project

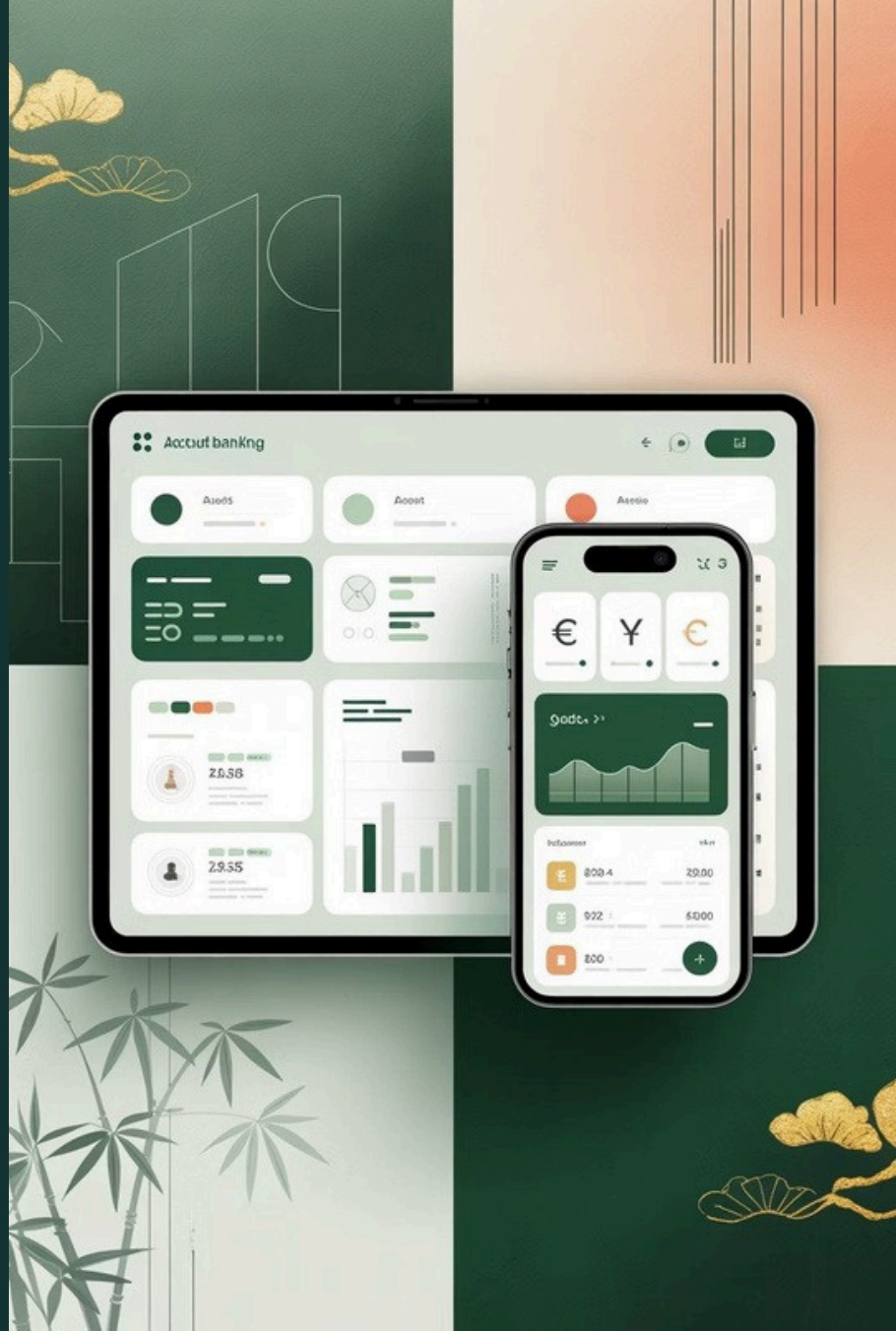

Group Members:

Arya Agrawal - 1262252003

Mithil Kale - 1262254135

Diya Varma - 1262252717

Aayush Khandelwal - 1262252807

# Project Objectives

To develop a menu-driven bank account simulation using the C programming language.

To implement basic banking operations such as deposit, withdrawal, and balance enquiry

To strengthen understanding of loops, conditional statements, and switch-case logic through a real-life application.

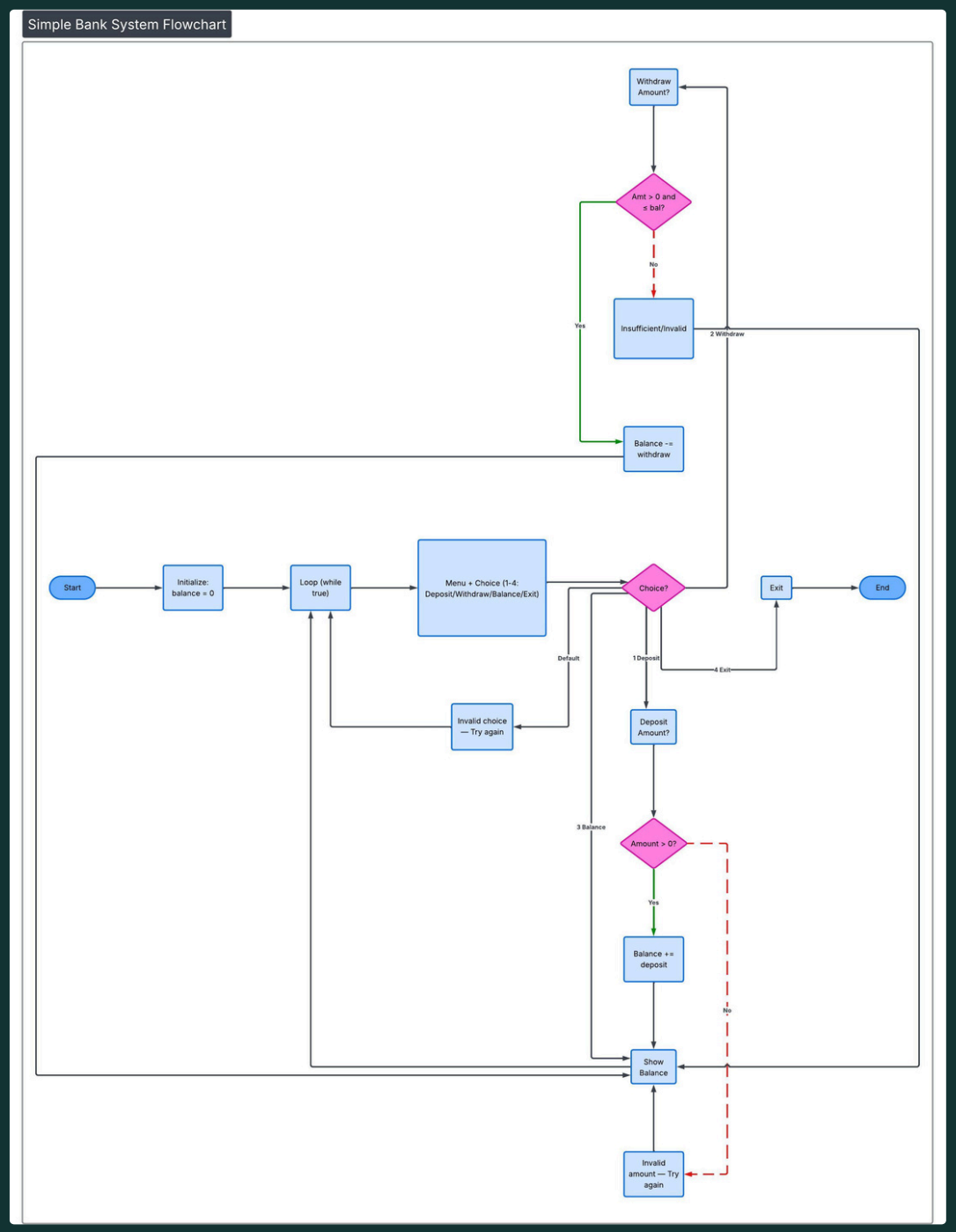
# Core Features

### Deposit Money
Addfunds toyouraccount with validation checks

### Withdraw Money
Remove funds whilst preventing overdrafts

### Check Balance
Viewcurrentaccount balance in real-time

# Program Algorithm




description of the algorithm

- **Program Initialization:** The flow begins by setting the starting account balance to **0.0** and preparing the system to accept user inputs.
- **Continuous Loop Logic:** The while (1) loop ensures the banking interface remains active, re-displaying the menu after every transaction until the user chooses to exit.
- **Choice-Based Navigation:** A central decision diamond (the switch case) directs the flow to specific modules based on the user's selection: **Deposit**, **Withdraw**, **Check Balance**, or **Exit**.
- **Integrated Error Checking:** Every transaction includes a validation step; for example, it checks if a deposit is a positive number or if a withdrawal exceeds the current available balance before updating the account.
- **Finalization:** Choosing "Exit" triggers a farewell message and breaks the loop to securely reach the **END** of the program.

# Program code

## Code

```c
#include <stdio.h>

int main()
{
    int choice;
    float balance = 0.0, amount;

    printf("=== Welcome to Simple Bank ===\n");

    while (1)
    {
        printf("\n1. Deposit Money");
        printf("\n2. Withdraw Money");
        printf("\n3. Check Balance");
        printf("\n4. Exit");
        printf("\nEnter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                printf("Enter amount to deposit: ");
                scanf("%f", &amount);

                if (amount > 0)
                {
                    balance += amount;
                    printf("Deposit successful!");
                }
                else
                {
                    printf("Invalid amount!");
                }
                break;

            case 2:
                printf("Enter amount to withdraw: ");
                scanf("%f", &amount);

                if (amount > 0 && amount <= balance)
                {
                    balance -= amount;
                    printf("Withdrawal successful!");
                }
                else
                {
                    printf("Insufficient balance or invalid amount!");
                }
                break;

            case 3:
                printf("Your current balance is: %.2f", balance);
                break;

            case 4:
                printf("Thank you for using our bank!");
                return 0;

            default:
                printf("Invalid choice! Try again.");
        }
    }
}
```

## Output

```
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 1
Enter amount to deposit: 5000
Deposit successful!
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 2
Enter amount to withdraw: 4500
Withdrawal successful!
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 3
Your current balance is: 500.00
1. Deposit Money
2. Withdraw Money
3. Check Balance
4. Exit
Enter your choice: 4
Thank you for using our bank!
```

# Code Description

This programactsasa **virtualbankteller** that manages a single account balance through an interactive menu. It allows users to deposit funds, withdraw money with built-in safety checks to prevent overdrawing, and view their current balance in real-time.

- **Interactive Menu:** Provides a simple, perpetual interface for users to choose between four distinct banking actions.
- **Balance Management:** Real-time tracking of funds with precise decimal accuracy for every transaction.
- **Smart Validation:** Built-in safeguards to block invalid entries and prevent users from withdrawing more than they own.

# Switch-Case Control Structure

## Menu Options

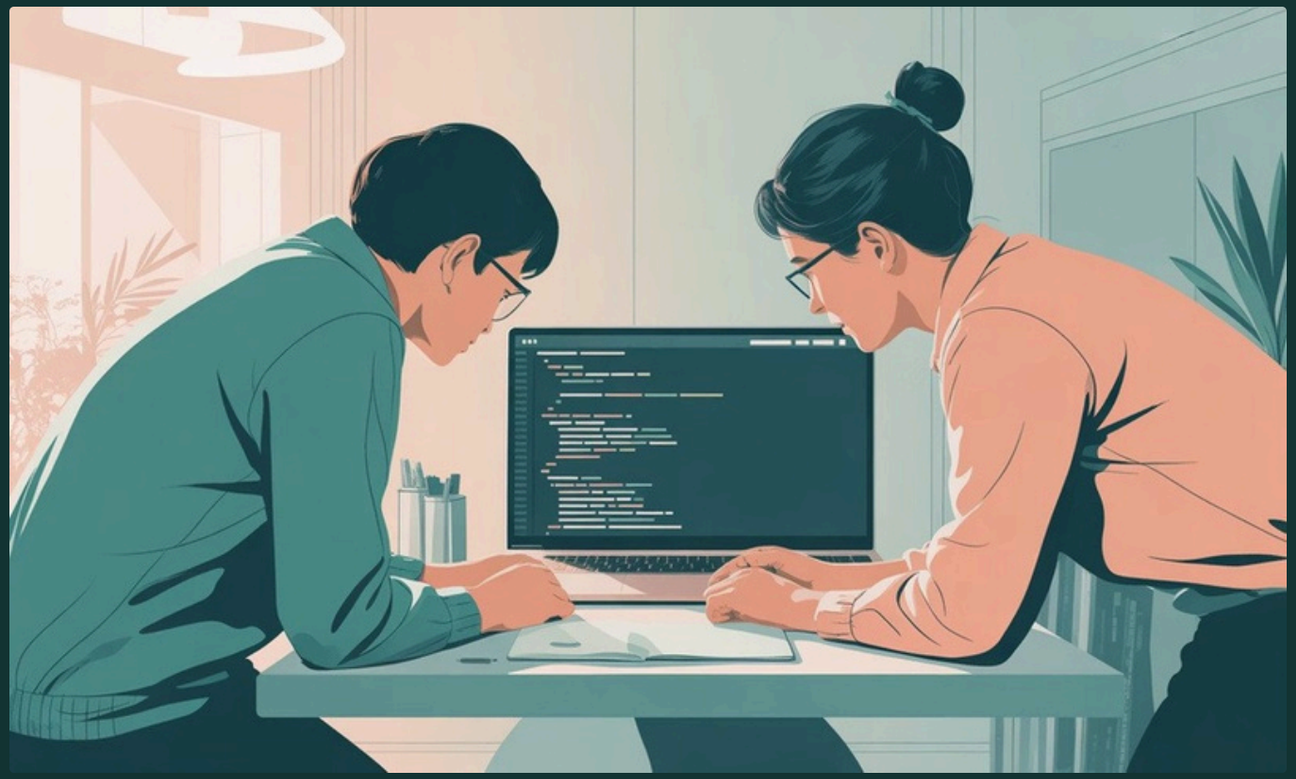
> The programme uses a switch statement to handle four distinct cases, making the code clean and efficient for menu navigation.

- **Case 1:** Deposit operation
- **Case 2:** Withdrawal operation
- **Case 3:** Balance enquiry
- **Case 4:** Exit programme
- **Default:** Invalid input handler

### Why Switch Over If-Else?

Switch statements provide cleaner syntax for multiple discrete options and improve code readability when handling menu-driven programmes.

# Learning Outcomes



## Skills Demonstrated

This mini-project successfully demonstrates fundamental C programming concepts through practical application.

- Control structures (loops and conditionals)
- User input handling with scanf
- Variable manipulation and arithmetic
- Logical condition building
- Menu-driven programme design

**Next Steps:** Consider adding features like transaction history, multiple accounts, or file storage for persistence.

# Thank You

We welcome your questions and feed-back!